CuyCash · Avance de Proyecto Final 3 (APF3)

Curso: Integrador II: Software · **Docente:** Dr. Carlos R. P. Tovar **Producto:** CuyCash, app móvil de banca digital (Perú) · **Versión evaluada:** v2

Repositorio	https://github.com/Los-hijos-de-Rene/CuyCashUTP
Producción (API)	https://cuycashutp.onrender.com · `GET /health` devuelve el commit desplegado
Tablero de trabajo	https://github.com/orgs/Los-hijos-de-Rene/projects/1
Commit evaluado	PENDIENTE (copiar de `/health` el día de la entrega)
Fecha de corte	PENDIENTE

Equipo

Integrante	Aporte en APF3	Capítulo
Jair Conislla	Observaciones APF2, evidencia automática en CI, contrato app↔API, consolidación	A, anexos
Gian Piero	Pruebas funcionales y UAT	B
Jeremy	Pruebas de integración con sistemas externos	C
Jheampierre	Suite automatizada de usabilidad	D
Jair Abdel Linares	Informe de usabilidad con usuarios	D.1
Andersson	Validación en la nube y coexistencia	E, C

Cómo leer este documento

Cada capítulo responde, en orden y con su numeración, a los artefactos que pide la guía de APF3. Todo resultado lleva el comando que lo produjo y su fecha. Lo que no se pudo probar se declara en «Observaciones» de cada capítulo.

A. Levantamiento de observaciones del APF2

Observación recibida en la sustentación: usar bien la metodología de trabajo con GitHub Projects, asignando a cada integrante sus tareas y su prioridad por niveles.

#	Requisito	Antes	Acción	Evidencia	Estado
1	Cada tarea con responsable	35 de 35 ítems sin *Assignee*	Un dueño por ítem, revisor distinto; tarjetas APF3 #96–#113 con dueño	Captura de la vista *Table* (columna Assignees) — PENDIENTE	🟡
2	Prioridad por niveles	Campo *Priority* sin opciones	Campo **Nivel de prioridad** (P0 Crítica / P1 Alta / P2 Media) en los 43 ítems comprometidos, más número único de orden	Captura agrupada por *Nivel de prioridad* — PENDIENTE	✅
3	Todos avanzan	294 de 294 commits de una cuenta	Spec por integrante; límite de 2 en curso; PR aprobado por otra persona	`git shortlog`, PR por autor — PENDIENTE (al entregar)	🟡
4	Estado, tamaño, estimación, sprint	Campos vacíos	*Size* en todo; *Estimate* en días (S 1, M 2½, L 5); *Sprint* 5 y 6; 16 tarjetas APF3 en Sprint 6	Captura de *Roadmap* — PENDIENTE	🟡

Porcentaje levantado: PENDIENTE (se declara al entregar; no se afirma 100 % con filas 🟡).

Detalle y comandos para recalcular la evidencia: `docs/apf3/capitulo-a-observaciones.md`.

B. Pruebas funcionales automatizadas

Responsable: Gian Piero Gonzales · **Tarjetas:** #96 y #97 · **Corte:** 02/11/2026 · **Commit probado:** `44a9c36`.

El objetivo es verificar completitud, corrección e idoneidad funcional con criterios trazables. Se distingue entre una prueba que termina en verde y un criterio de aceptación realmente cumplido: ejecutar líneas no demuestra por sí solo que el producto entregue toda la función solicitada.

B.1 Frameworks utilizados y justificación

Capa	Framework	Justificación técnica
API	`pytest 8.3.3`, `pytest-asyncio` y `httpx`	Permiten probar servicios asíncronos y contratos HTTP completos, incluyendo código, cuerpo y efectos en base. Los fixtures aíslan cada caso y hacen repetible la preparación de titulares, cuentas y sesiones.
Persistencia	SQLAlchemy asíncrono con SQLite local y PostgreSQL 16 en CI	SQLite da realimentación rápida; PostgreSQL 16 valida restricciones, bloqueo de filas y concurrencia con el mismo motor previsto para producción. Una prueba local verde no sustituye el job PostgreSQL.
App	`flutter_test` y `bloc_test`	`flutter_test` valida widgets, navegación y accesibilidad; `bloc_test` comprueba estados, transiciones y errores sin depender del renderizado. Los contratos de repositorio comparan implementaciones `Memory*` y `Http*`.
Cobertura	`pytest-cov/coverage.py` y `flutter test --coverage`	Miden líneas ejecutadas y detectan módulos sin ejercicio. La cobertura se interpreta junto con aserciones y RTM; no equivale a ausencia de defectos.
Evidencia	JUnit XML y Cobertura XML	Son formatos reproducibles y consumibles por CI. Conservan caso, duración, estado y cobertura sin transcribir resultados manualmente.

B.2 Listado trazable de 20 criterios

La selección cubre las cinco HU operativas principales de identidad, consulta, transferencias y contabilidad. El resultado de la última columna es el **veredicto del CA**, no solo el color del test. 🟡 indica que la parte citada pasó, pero queda una condición funcional sin implementar o medir.

ID	Requisito / criterio	Precondición	Pasos	Resultado esperado	Archivo y nombre exacto de la prueba
HU03-CA1	Ingreso facial <1.5 s	Usuario registrado	Intentar autenticación facial y medir	Sesión en menos de 1.5 s	No existe prueba ni login facial
HU03-CA2	Huella o PIN alternativo	Biometría vinculada	Autenticar con credencial válida	Sesión segura creada	`test_biometria.py::test_activar_y_entrar_con_huella`
HU03-CA3	Bloqueo tras fallos y aviso	DNI registrado	Fallar el PIN repetidamente	Bloqueo y notificación	`test_registro_y_login.py::test_tres_fallos_bloquean_el_dni_y_ni_el_pin_co`
HU03-CA4	Cierre por inactividad	Sesión abierta	Superar tiempo inactivo	Token/sesión revocados	No existe temporizador ni prueba
HU05-CA1	Cuentas propias, tipo, número enmascarado y saldo	Dos titulares	Consultar `/v1/accounts`	Solo cuentas propias con datos permitidos	`test_cuentas_y_movimientos.py::test_solo_devuelve_las_cuentas_propias`
HU05-CA2	Saldo contable y disponible con retenciones	Cuenta con fondos retenidos	Consultar saldos	Ambos saldos reflejan la retención	No existe modelo de retenciones
HU05-CA3	Movimientos de la cuenta seleccionada	Varias cuentas con operaciones	Consultar historial por cuenta	No mezcla movimientos	`test_cuentas_y_movimientos.py::test_el_historial_no_mezcla_movimientos_de`
HU05-CA4	Exportación PDF/Excel	Historial disponible	Solicitar exportación	Archivo descargable coherente	No existe exportación
HU06-CA1	Transferencia propia inmediata y sin costo	Dos cuentas PEN del titular	Enviar S/10.00	Débito y crédito exactos	`test_envio_por_cuenta.py::test_entre_cuentas_propias_se_puede`
HU06-CA2	Interbancaria CCI/CCE	CCI válido y red disponible	Confirmar envío	Titular validado y orden emitida	No existe conector interbancario
HU06-CA3	Rechazo sin fondos sin afectar saldos	Saldo menor al monto	Intentar enviar	`INSUFFICIENT_FUNDS`, cero asientos	`test_transferencias.py::test_sin_saldo_suficiente_responde_insufficient_f`
HU06-CA4	Comprobante completo	Transferencia confirmada	Consultar/compartir constancia	Fecha, monto, cuentas y operación	`receipt_card_test.dart::muestra monto, contraparte, motivo, estado, fecha`
HU17-CA1	Validar saldo antes de ejecutar	Cuenta con saldo insuficiente	Publicar débito	No se escribe nada	`test_motor_de_asientos.py::test_sin_saldo_no_escribe_nada`
HU17-CA2	Bloquear fondos hasta confirmar	Cuenta leída por dos operaciones	Bloquear y releer	Se usa el saldo actual bajo bloqueo	`test_motor_de_asientos.py::test_una_cuenta_ya_cargada_se_relee_tras_el_bl`
HU17-CA3	Idempotencia	Transferencia ya confirmada	Repetir la misma clave	Devuelve la original sin nuevo cargo	`test_motor_de_asientos.py::test_repetir_la_clave_devuelve_la_original_sin`

ID	Requisito / criterio	Precondición	Pasos	Resultado esperado	Archivo y nombre exacto de la prueba
HU17-CA4	Procesamiento <200 ms	Entorno de carga controlado	Medir transacción p95/p99	Duración menor a 200 ms	No existe prueba de rendimiento
HU18-CA1	Débito = crédito	Operación autorizada	Contabilizar transferencia	Sumas iguales por transacción	`test_libro_mayor.py::test_una_transferencia_deja_los_debitos_iguales_a_lo`
HU18-CA2	UUID por asiento	Transacción con dos asientos	Persistir y consultar IDs	IDs propios, válidos y distintos	`test_motor_de_asientos.py::test_cada_asiento_y_su_transaccion_llevan_un_u`
HU18-CA3	Atomicidad, sin asiento parcial	Entrada inválida o saldo insuficiente	Intentar publicar	Rollback completo	`test_motor_de_asientos.py::test_entradas_invalidas_revientan_antes_de_toc`
HU18-CA4	Cierre contable diario	Día de negocio cerrado	Ejecutar conciliación	Diferencias detectadas y auditadas	No existe proceso de cierre

Corrección numérica obligatoria

Caso	Control verificado	Resultado 02/11/2026
Dinero sin `double`	Montos HTTP y persistencia son enteros de céntimos; más de dos decimales se rechazan	✅ `test_un_monto_que_no_es_entero_de_centimos_no_mueve_dinero`
Envío de S/125.50	UAT-1 usa `12550` céntimos y verifica ambos saldos	✅ `test_uat1_enviar_dinero_a_otro_titular_y_que_ambos_lados_cuadren`
Partida doble	Total débito = total crédito por ID de transacción	✅ `test_una_transferencia_deja_los_debitos_iguales_a_los_creditos`

B.3 Ejecución reproducible

Comando ejecutado desde `services/api` en Windows 11, Python 3.12.10:

```
.\.venv\Scripts\python.exe -m pytest -q --cov=app --cov-report=term `
  --cov-report=xml:..\..\docs\apf3\evidencias\gian-piero\cobertura-backend.xml `
  --junitxml=..\..\docs\apf3\evidencias\gian-piero\pytest.xml
```

Resultado final: **428 aprobadas, 0 fallidas, 4 omitidas y 22 advertencias** en **210.76 s**. Las omisiones dependen de capacidades no disponibles en el host local, entre ellas pruebas específicas de PostgreSQL y `time.tzset` de POSIX; el job CI PostgreSQL debe confirmar las primeras.

La primera corrida detectó tres incompatibilidades portables en Windows: codificación de `schema.sql`/salida del generador DDL y uso incondicional de `time.tzset`. Se fijó UTF-8 explícito y se convirtió el caso POSIX en una omisión declarada. La repetición dirigida dio **27 aprobadas, 1 omitida** y la repetición completa produjo el resultado verde anterior.

Evidencias reproducibles:

- `docs/apf3/evidencias/gian-piero/pytest.xml`
- `docs/apf3/evidencias/gian-piero/cobertura-backend.xml`
- `docs/apf3/evidencias/gian-piero/pytest-resumen.txt`
- `docs/apf3/evidencias/gian-piero/entorno-ejecucion.txt`
- `docs/apf3/evidencias/gian-piero/ejecucion-backend.png`

La app se ejecutó con la misma versión fijada en CI, Flutter 3.41.7:

```
flutter analyze
flutter test --coverage --coverage-path=reports/lcov.info `
  --file-reporter=json:reports/flutter-tests.json
```

`flutter analyze` terminó con **0 problemas**. La suite móvil terminó con **905 aprobadas, 0 fallidas y 2 omitidas**. Además, `core_kernel` aprobó 24/24 y `design_system` 12/12. Los reportes reproducibles se guardaron como `flutter-tests.json`, `cobertura-app.lcov` y `flutter-resumen.txt` dentro de la carpeta de evidencias de Gian Piero. La imagen `ejecucion-flutter.png` resume visualmente esos mismos reportes y declara su procedencia.

B.4 Cobertura

Componente	Líneas/sentencias	Sin ejecutar	Cobertura	Estado
Backend total	1,916	72	**96 %**	Medido localmente
`services/ledger.py`	73	1	99 %	Crítico, medido
`routers/transfers.py`	90	1	99 %	Crítico, medido
`services/loan_schedule.py`	103	1	99 %	Cálculo puro, sin API todavía
`routers/kyc.py`	72	9	88 %	Requiere ampliar casos
`services/notifier.py`	31	13	58 %	Principal brecha de backend
App Flutter	8,434	1,942	**76.97 %**	Medido con Flutter 3.41.7

La cobertura móvil se calculó desde el LCOV real: 6,492 líneas ejecutadas de 8,434 líneas encontradas en 254 archivos. El reporte conserva el detalle por archivo para que CI o un visor LCOV puedan reproducir el porcentaje.

B.5 Métricas ISO/IEC 25023 y cumplimiento

Métrica	Fórmula	Resultado al 02/11/2026	Interpretación
Casos backend	aprobados / fallidos / omitidos	428 / 0 / 4	432 recolectados
Casos app	aprobados / fallidos / omitidos	905 / 0 / 2	907 recolectados
Paquetes compartidos	aprobados / fallidos	36 / 0	24 `core_kernel` + 12 `design_system`
Cumplimiento de ejecución	aprobados ÷ recolectados	**99.6 %** (1369/1375)	Supera meta de 90 %; las 6 omisiones no se ocultan
Corrección ejecutada `Y`	verdes ÷ ejecutados	**100 %** (1369/1369)	No hubo fallo entre los casos que sí corrieron
Completitud funcional `X`	CA implementados y automatizados ÷ CA MVP	**12.5 %** (12/96)	Los CA parciales y los oráculos previos al código no cuentan
Cobertura backend	sentencias ejecutadas ÷ sentencias	**96 %**	No implica ausencia de defectos
Cobertura app	líneas cubiertas ÷ líneas	**76.97 %** (6492/8434)	Reporte LCOV reproducible

Observaciones y limitaciones

1. La ejecución local usa SQLite; bloqueo de fila e idempotencia concurrente deben reconfirmarse en PostgreSQL 16 mediante CI.
2. La suite Flutter se ejecuta en el host de pruebas y sustituye sensores con dobles controlados; no demuestra cámara, huella ni rendimiento en Android.
3. El SLA transaccional de 200 ms y el ingreso de 1.5 s no se midieron con una herramienta de carga; permanecen como CA no cumplidos.
4. QR y préstamos tienen oráculos previos en `docs/casos-aceptacion-qr-prestamos.md`, pero las API/pantallas necesarias para #91 aún no existen en este commit. El cálculo puro del cronograma no acredita por sí solo HU09/HU12.
5. Un 96 % de cobertura solo indica líneas recorridas. La calidad depende de los oráculos, límites, datos negativos y revisión de código.

B.6 UAT y acta de validación (#97)

Las pruebas automatizadas UAT-1..3 del backend están verdes dentro de la suite. La validación visual completa sigue **pendiente** porque Docker Desktop no está activo y Flutter/emulador no están disponibles en este host. No se firma un acta manual sin recorrer la app real. Para cerrar #97 faltan:

- levantar API y app local;
- sembrar Ana/Luis;
- ejecutar cada paso de `docs/verificacion-manual.md`;
- completar `docs/vyv.md` §5 con capturas, discrepancias y firma del revisor.

Esta separación evita presentar una prueba HTTP automatizada como si fuera una aceptación de usuario en emulador.

C. Pruebas de integración automatizadas *(Jeremy #100, #101 · Andersson #103 · contrato: Jair C. #111)*

C.1 Sistemas externos a integrar

Sistema	Cómo se integra	Riesgo de interoperabilidad	Mitigación
Microservicio KYC (`CuyCashKYC`)	Proxy de `services/api` (`/v1/kyc/*`); la clave vive en el backend	Se duerme en plan gratuito; cambio de contrato; red	Reintento solo si la petición no llegó (502/503/504); clave rechazada → 503; lista blanca de rutas
PostgreSQL (Neon)	`asyncpg` + SQLAlchemy	Neon suspende el cómputo y cierra conexiones	`pool_pre_ping=True`
Render (hosting)	Docker, `$PORT`, `/health`	Arranque en frío	Despliegue solo por Actions; health check; rollback
Telegram (OTP de demostración)	`httpx`, timeout 10 s	Cae el servicio	Un fallo no tumba el flujo; reenvío
ML Kit / `local_auth`	Plugins de Flutter	Cambian con el sistema operativo	Detrás de interfaces con doble `Memory*`
GitHub Actions	Workflows	Cambio de versión de una acción	Acciones fijadas por versión mayor

Fuente: `docs/compatibilidad-y-coexistencia.md` §1.

C.2 Frameworks

`pytest` + `pytest-asyncio` + `httpx.MockTransport` (servicio externo simulado); PostgreSQL 16 real en CI; `flutter test` con los repositorios `Http*` reales contra la API levantada (contrato).

C.3 Casos de prueba de interoperabilidad

- Proxy KYC: 9 casos existentes en `services/api/tests/test_proxy_kyc.py`.
- Contrato app ↔ API real: INT-CT-01..10 en `apps/mobile/contract_test/backend_contract_test.dart`.
- Telegram, KYC (timeouts, JSON inválido), base de datos y OpenAPI: PENDIENTE (Jeremy, `docs/apf3/spec-jeremy-integracion.md`).

C.4 Capturas de ejecución · C.5 Reportes de resultados · C.6 Métricas y observaciones

PENDIENTE — capturas de `pytest -v` y del job *Contrato app ↔ API*; `junit.xml`; tabla de casos y % de cumplimiento; límites (el KYC real no corre en CI).

D. Evaluación de la usabilidad *(Jheampierre #98, #99 · Jair A. Linares #104)*

D.1 Informe de evaluación de usabilidad

PENDIENTE — heurísticas de Nielsen, 5 usuarios, SUS, tiempos de tarea (`docs/usabilidad.md`).

D.2 Frameworks

`flutter_test` con las *guidelines* de accesibilidad (`androidTapTargetGuideline`, `labeledTapTargetGuideline`, `textContrastGuideline`), `bloc_test`, dobles `Memory*`. Límite: no mide aprendizaje humano (eso es D.1).

D.3 Casos de prueba de interfaz y usabilidad

Modelo existente: `apps/mobile/test/usabilidad/login_usabilidad_test.dart` (UX-01..04). Resto de pantallas: PENDIENTE.

Hallazgo ya resuelto (UX-02): el botón «volver» del login no tenía etiqueta para lectores de pantalla; lo detectó la prueba y se corrigió con `tooltip` (`commonBack`).

D.4 Capturas · D.5 Reportes · D.6 Métricas y observaciones

PENDIENTE — corrida de la suite, tabla de resultados por caso, % de cumplimiento, hallazgos y su corrección.

E. Validación en la plataforma Cloud (Andersson #102)

Ensayo del 2026-11-04 01:36–01:39 UTC (03/11 en Lima). URL: https://cuycashutp.onrender.com. Evidencias reales en `../evidencias/andersson/`.

Resultado contra Render + Neon

Verificación	Esperado	Obtenido
`/health`	HTTP 200 y commit	200; `d74d66326ee264f7e68ef550b741d107916b6076` (merge PR #64)
`/health/db`	PostgreSQL disponible	200, `status=ok`; latencia interna 340,9 ms en segunda muestra
Humo existente	0 fallos	0 fallos; `smoke-production.txt`
HTTP → HTTPS	301	301
Sesión ausente / token inventado	401	401 en ambos casos
Cabeceras	HSTS, nosniff, DENY, no-store en API	Presentes; `production-readonly.json`
DNI de 7 dígitos	422	422 en `/v1/auth/authenticate`, sin autenticar ni registrar usuario
Alta UAT ficticia	201	**403 KYC_REQUIRED**, ensayo detenido
UAT-1/UAT-2 en nube	Envío y rechazo sin saldo	**Bloqueados por KYC**; no se declararon aprobados
Cuadre SQL en Neon	Cero diferencias	**No ejecutado**: no hay conexión/credencial Neon disponible
APK de producción → nube	Recorrido y constancia	**No ejecutado**: no hay Flutter, SDK Android completo ni teléfono autorizado conectado
CD de main actual	Commit actual desplegado	Último CD consultado pendiente; no coincide con commit servido

`/health` en caliente tomó 270,554 ms (humo); segunda muestra HTTP 295,53 ms. Son muestras puntuales con latencia de red; no equivalen a p95 ni al SLA del motor transaccional. No se midió un arranque en frío porque el servicio ya estaba respondiendo. Los JSON incluyen tiempos y estados exactos de cada petición.

Ensayo funcional y límites

El usuario autorizó explícitamente crear dos usuarios ficticios y operaciones simuladas en producción. `scripts/uat_prod.py` requiere `--allow-writes` y, para el host de Render, `--production-approved`. Nunca imprime PIN ni tokens. Reserva **solo dentro del proyecto** los identificadores sintéticos 00001021 y 00001022, nombres de ensayo y correos bajo example.org. No son documentos validados por RENIEC. Si ya existen, el script se detiene; no toma una cuenta existente ni suplanta a su titular. No se deshabilitó KYC ni se usó biometría. La primera alta fue rechazada; no hubo depósito ni envío en producción.

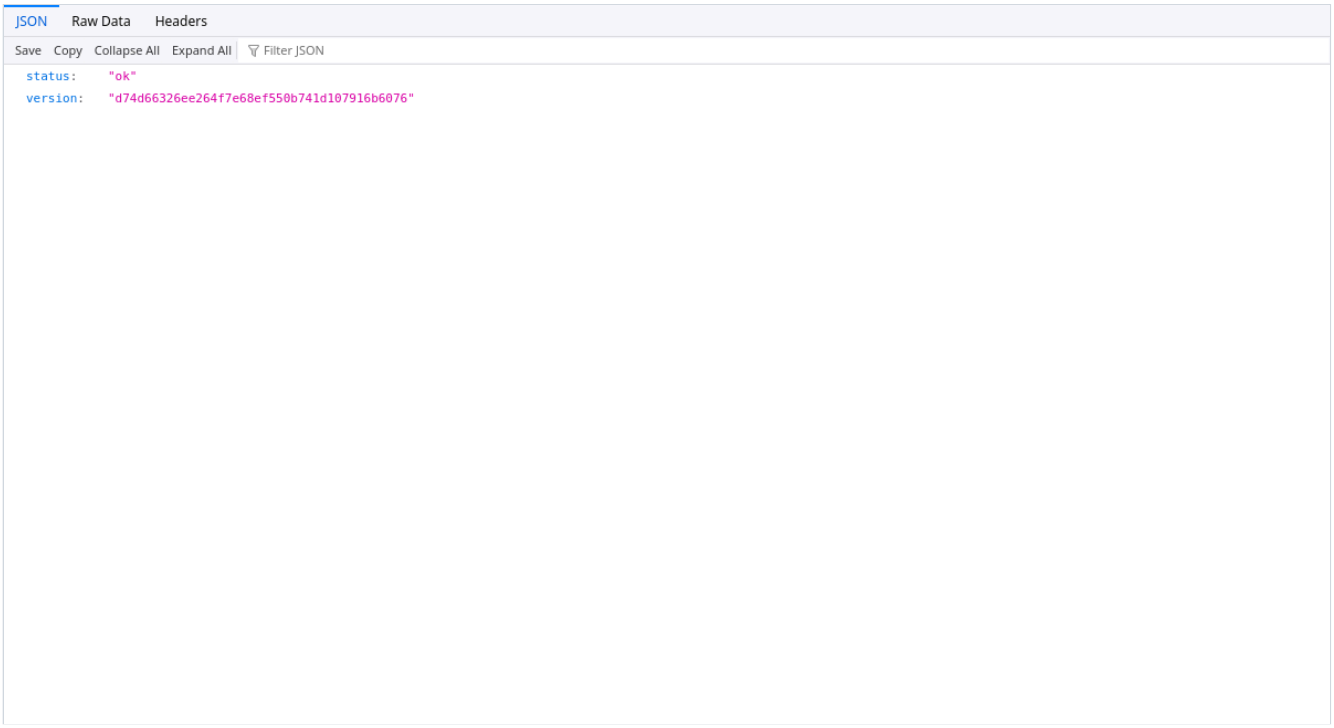
Como verificación adicional, ambos UAT se ejecutaron por HTTP contra Docker y PostgreSQL 16 **local**, con base de desarrollo separada de las bases pytest. Resultado `uat-local.json`: aprobado. UAT-2 primero: depósito de 5000 céntimos, dos intentos de 50000 rechazados, saldos y movimientos conservados. Luego se completa el saldo a 50000 y UAT-1 envía 12550: saldos finales 37450/12550; reintento 200 con el mismo ID de transacción. Las dos sesiones se revocaron. **Esta evidencia local no sustituye UAT de nube ni prueba Android.**

Pendientes de entrega

102 sigue parcial: requiere acordar con Jeremy una forma de ejecutar UAT con

identidades ficticias y KYC válido sin eludir controles (o una rama Neon de ensayo aprobada); obtener acceso de solo lectura para el cuadre SQL; completar el recorrido Android y disponer de la versión v2 efectivamente desplegada. Desplegar requiere autorización explícita y el procedimiento de migración y respaldo de PR #95. No se aprobó ni ejecutó un despliegue desde esta sesión. El capítulo se incluye mediante el generador PDF existente; el PDF versionado queda pendiente de regeneración por el responsable del informe tras revisión.

Captura real del navegador (headless, sin editar):



`ledger-local.txt` registra cero transacciones y cero cuentas descuadradas **en PostgreSQL local** después de UAT; no es una consulta a Neon.

Anexos

Enlaces a los scripts de automatización (repositorio)

Qué	Dónde
Pruebas del backend (API, seguridad, concurrencia)	`services/api/tests/`
UAT automatizados	`services/api/tests/test_aceptacion_uat.py`
Pruebas de la app (unitarias, bloc, widgets)	`apps/mobile/test/`
Usabilidad (UI)	`apps/mobile/test/usabilidad/`
Contrato app ↔ API	`apps/mobile/contract_test/`
Humo en producción	`services/api/scripts/smoke_prod.sh`
CI/CD	`.github/workflows/ci.yml`, `cd-backend.yml`, `rollback-backend.yml`, `seguridad.yml`
Reportes de CI	artefactos `reportes-backend`, `reportes-app`, `reporte-contrato` de cada corrida

Documentos de calidad

`docs/idoneidad-funcional.md` (RTM, ISO 25023) · `docs/vyv.md` (V&V, UAT) · `docs/compatibilidad-y-coexistencia.md` · `docs/plan-de-pruebas.md` · `docs/seguridad.md` · `docs/despliegue.md`.